Sir Syed CASE Institute of Technology

SUBMITTED BY : SPOGMAI JAN

ROLL NO : 2430 - 0081

COURSE : AI LAB

DATE OF SUBMISSION : 12/03/2026

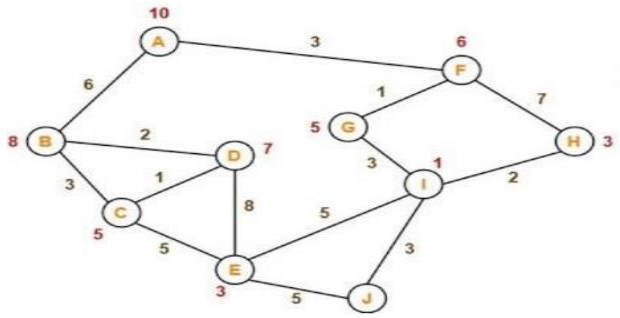
DEPARTMENT : COMPUTER SCIENCE

SUBMITTED TO : MAM WARDAH

ASSIGNMENT NO :05

Task :

1. Implement the A* search algorithm to find the path from State A to J. Find the Optimal Path's Order of Expansion and the Total Cost.



Code:

```
graph = {
    'A': {'B': 6, 'F': 3},
    'B': {'A': 6, 'C': 3, 'D': 2},
    'C': {'B': 3, 'D': 1, 'E': 5},
    'D': {'B': 2, 'C': 1, 'E': 8},
    'E': {'C': 5, 'D': 8, 'I': 5, 'J': 5},
    'F': {'A': 3, 'G': 1, 'H': 7},
    'G': {'F': 1, 'I': 3},
    'H': {'F': 7, 'I': 2},
    'I': {'E': 5, 'G': 3, 'H': 2, 'J': 3},
    'J': {'E': 5, 'I': 3}
}

heuristics = {
    'A': 10, 'B': 8, 'C': 5, 'D': 7, 'E': 3,
    'F': 6, 'G': 5, 'H': 3, 'I': 1, 'J': 0
}

def A_star_search(start, goal):
    open_list = [start]
    g_cost = {start: 0}
    parent = {start: None}

    while open_list:
        current = min(open_list, key=lambda x: g_cost[x] + heuristics[x])

        if current == goal:
            path = []
            while current is not None:
                path.append(current)
                current = parent[current]

            path.reverse()
            print("Path found:", " → ".join(path))
            print("Total cost:", g_cost[goal])
            return

        open_list.remove(current)

        for neighbor, cost in graph[current].items():
            temp_g = g_cost[current] + cost

            if neighbor not in g_cost or temp_g < g_cost[neighbor]:
                g_cost[neighbor] = temp_g
                parent[neighbor] = current


               if neighbor not in open_list:
                   open_list.append(neighbor)
```

Output:

```
x = input("Enter the start node: ")
y = input("Enter the goal node: ")

A_star_search(x, y)
```

```
Enter the start node: B
Enter the goal node: J
Path found: B → C → E → J
Total cost: 13
```

==========